Smart Home

Integrantes:

Iván Darío Hernández

Estefania Moreno Reyes

Edson Jaime Pinto Abreo

Daniel José Suarez Gonzales

Jesús David Arias Estupiñán

A198: Patrones de Software

Facultad de Ciencias Naturales e Ingeniería, Ingeniería de Sistemas

Unidades Tecnológicas de Santander

11 de octubre de 2025

Bucaramanga-Santander

Contenido

Resumen....4

Contexto....5

Introducción....6

Descripción del Problema....7

Justificación....8

Objetivos....9

Objetivo General....9

Objetivos Específicos....9

Desarrollo....10

Código....11

Estructura....11

Sketch....12

Commands....13

Factory....16

State....17

Strategy....18

TextToSpeech....18

Main....21

Serial Manager....29

Montaje Físico 32

Materiales 32

Procedimiento 32

Prueba de Funcionamiento 38

Conclusiones 39

Referencias 41

Resumen

El presente informe muestra el desarrollo de un prototipo de Hogar Inteligente (Smart Home) que integra los patrones de diseño aplicados a la automatización de un entorno doméstico, asimismo se implementan los patrones Command, Factory, Observer y State.Ademas, se utilizaran componentes físicos como el Arduino UNO o ESP32, LEDs, resistencias, protoboard y un servomotor, con el fin de simular un hogar automatizado de manera accesible y didáctica. El proyecto busca demostrar cómo el uso de patrones de diseño favorece la escalabilidad, flexibilidad y mantenibilidad de las soluciones de software en el ámbito de la domótica.

Contexto

Nombre de la Empresa: Soluciones Digitales Daniel Tomas

Sector: Tecnología –Automatización y Domótica

Descripción de la Empresa

Soluciones Digitales Daniel Tomas es una empresa innovadora que se dedica al desarrollo de soluciones tecnológicas orientada a la automatización del entorno domestico e industrial, su enfoque principal se centra en la creación de sistemas de Hogares Inteligentes (Smart Home) por medio de la integración de hardware accesible como el Arduino y el ESP32 junto con arquitecturas de software basadas en buenas prácticas de ingeniería, como los patrones de diseño.

La empresa busca acercar la tecnología a los usuarios de forma didáctica, escalable y sostenible, brindando productos y prototipos que no solo funcionen a nivel físico, sino que también estén respaldados por una lógica de programación robusta, adaptable a las necesidades futuras al cliente.

Soluciones Digitales Daniel Tomas promueve la investigación y la formación académica, estableciendo alianzas con instituciones educativas para el desarrollo de proyectos tecnología aplicados, como parte del compromiso con los ingenieros capacitados en IoT, automatización y software de calidad.

Introducción

En estos últimos años, los Hogares Inteligentes (Smart Homes) se han convertido en una tendencia cada vez más importante gracias a los avances tecnológicos, la conectividad y el internet de las cosas (IoT). Estos sistemas permiten automatizar actividades cotidianas como encender a apagar luces, abrir y cerrar puertas, controlar electrodomésticos o reforzar la seguridad del hogar brindándonos comodidad y eficiencia energética y una mayor tranquilidad para los usuarios.

Cabe destacar que, no se trata simplemente de instalar dispositivos electrónicos, sino que detrás de un Hogar Inteligente debe tener una arquitectura de software que sea flexible, escalable y fácil de mantener, por lo que aquí es donde entran en juego los patrones de diseño de software el cual ayudan a organizar el código, facilitar su reutilización permitir que el sistema pueda crecer con nuevas funciones sin que afecte su funcionamiento.

Este proyecto propone a los estudiantes de Ingeniería de sistemas la construcción de un prototipo de Smart Home aplicando los diferentes tipos de patrones de diseño como el patrón Command para el control de luces, el Factory para la creación dinámica de dispositivos, el Observer para notificar a los usuarios los cambios de estado y el State para manejar las diferentes situaciones de una puerta. En cuanto a la parte física, se trabajará con materiales accesibles como un Arduino UNO o ESP32, LEDs, resistencias, un servomotor, un protoboard con los cuales se simulará en un entorno doméstico automatizado.

Descripción del Problema

En la actualidad, los sistemas de Hogares Inteligentes se han popularizado como una alternativa para mejorar la calidad de vida de las personas mediante la automatización de actividades básicas. A pesar de ello, la mayoría de los prototipos se enfocan únicamente en la conexión y funcionamiento físico de los dispositivos electrónicos dejando de lado la arquitectura de software en el cual permita garantizar un sistema escalable, flexible y fácil de mantener.

Esta limitación genera debilidades en el diseño, como la dificultad para integrar nuevos dispositivos, la complejidad en la gestión de eventos y la poca reutilización del código. Estos problemas se originan por la ausencia de una base de programación organizada que aproveche patrones de diseño.

En este sentido, el problema a solucionar es la falta de modelos que integren tanto la parte física como la parte lógica de control basada en patrones de diseño, lo que reduce el alcance académico y practico de los prototipos de Smart Home.

Para ello surge la siguiente pregunta de investigación:

¿Cómo puede la aplicación de patrones de diseño de software mejorar la escalabilidad, flexibilidad y mantenibilidad de un prototipo de Smart Home enfocado en el control de luces y puertas?

(Guailas, 2023)

Justificación

El desarrollo de este proyecto responde a la necesidad de construir prototipos de Hogares Inteligentes que no se limiten únicamente al funcionamiento físico de los dispositivos, sino que también integren una arquitectura de software robusta. La ausencia de esta integración genera sistemas poco escalables, difíciles de mantener y con baja capacidad de crecimiento, lo que limita tanto su aplicabilidad como su valor académico.

Al implementar los patrones de diseño como el Command, Factory, Observer y State permitirá organizar el código de manera más clara, reutilizable y flexible, garantizando que el prototipo pueda adaptarse a nuevas funcionalidades sin comprometer su estabilidad.

Esta solución no solo fortalece al aprendizaje practico de los estudiantes, sino que también aporta al campo de la domótica como disciplina en expansión.

La relevancia del proyecto radica en que contribuye a las líneas de investigación de las UTS relacionas con el Internet de las Cosas (IoT), la automatización y la ingeniería de software aplicada. Además, su viabilidad técnica y económica es alta debido al uso de materiales accesibles como Arduino, ESP32, LEDs y servomotores.

En consecuencia, resolver esta problemática permitirá generar un impacto académico, tecnológico y social, al formar profesionales capaces de diseñar soluciones innovadoras que respondan a las necesidades actuales y futuras en el ámbito de los Hogares Inteligentes.

(Flores, 2014)

Objetivos

Objetivo General

Desarrollar un prototipo de Smart Home enfocado en el control de luces y puertas, integrando patrones de diseño de software que permitan mejorar la escalabilidad, flexibilidad y mantenibilidad del sistema.

Objetivos Específicos

- Analizar los principales patrones de diseño de software aplicables a la construcción de sistemas de Hogares Inteligentes (Command, Factory, Observer y State).
- Implementar un prototipo físico de Smart Home utilizando dispositivos accesibles (Arduino, ESP32, LEDs y servomotores), que permita la automatización de luces y puertas de manera básica y funcional.
- Integrar los patrones de diseño seleccionados en la programación del prototipo, empleando Java para estructurar el software de forma organizada, escalable y reutilizable.
- Documentar el proceso de diseño, desarrollo y los resultados obtenidos, resaltando el aprendizaje adquirido y las ventajas de utilizar una arquitectura basada en patrones de diseño para futuros proyectos académicos y prácticos en domótica.

Desarrollo

Durante esta fase se implementó un sistema funcional en Arduino aplicando cuatro patrones de diseño orientados a objetos: Command, Factory, Observer y State.El objetivo fue simular el encendido y apagado de luces, así como el control de apertura y cierre de una puerta, notificando los cambios de estado al usuario.

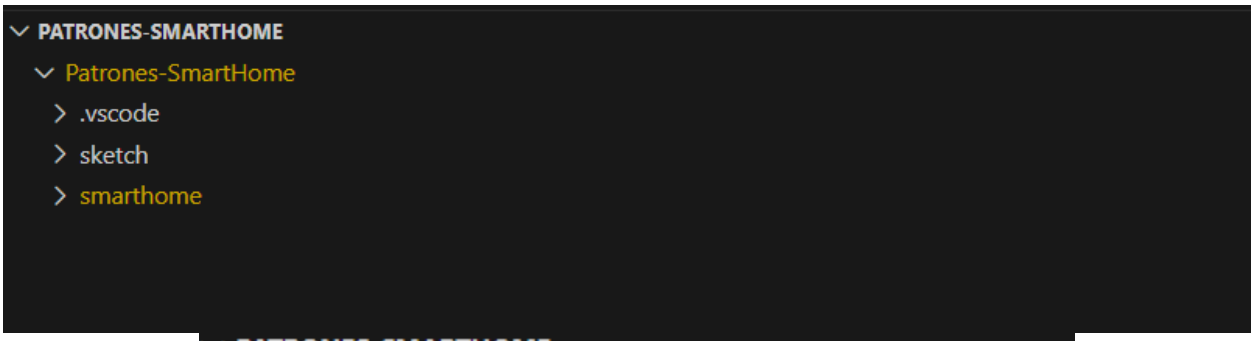
El sistema se desarrolló en la plataforma Wokwi utilizando un Arduino UNO, un LED y un servomotor, simulando el comportamiento de un entorno domótico básico.

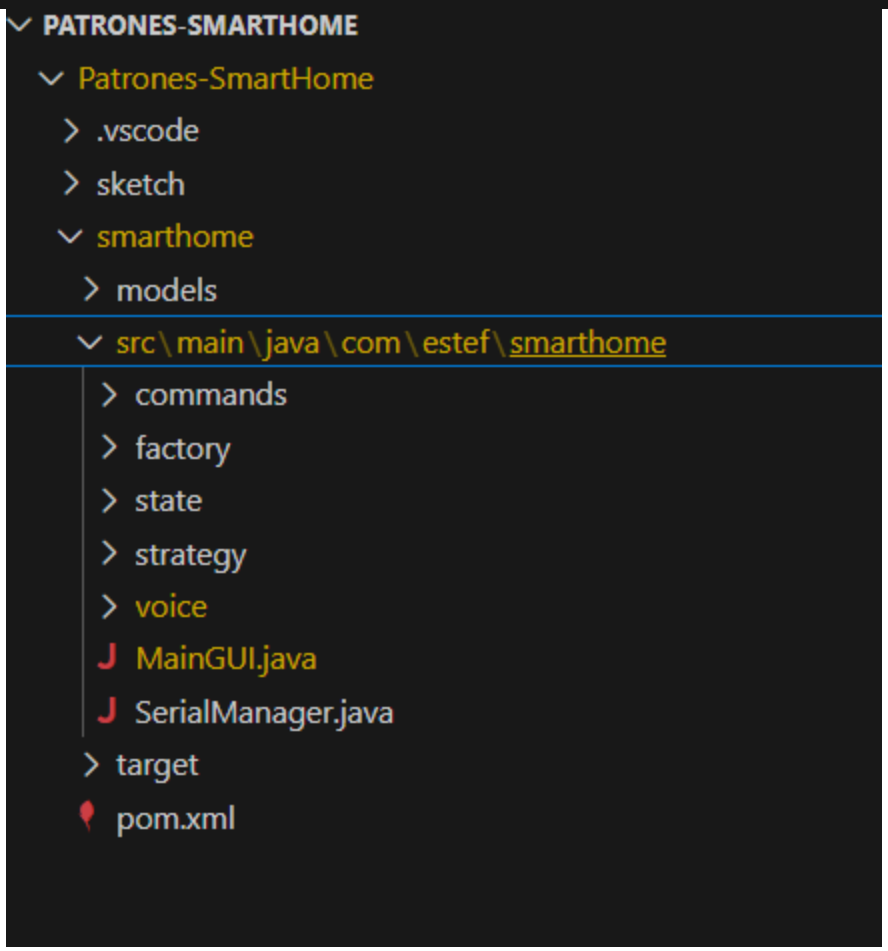
Cada patrón de diseño se aplicó de la siguiente manera:

- Command:Se utilizó para encapsular las acciones de encender y apagar una luz.
- Factory: Se implementó una fábrica (FactoryDispositivo) que permite crear dinámicamente distintos tipos de dispositivos (por ejemplo, una Luz o una Puerta).
- Observer: El patrón Observer se aplicó para notificar al usuario cada vez que un dispositivo cambia su estado.La clase Usuario recibe y muestra los mensajes de notificación a través del monitor serial.
- State:El control de la puerta se maneja mediante un patrón de estados (Abierta y Cerrada).

Código

Estructura





Sketch

```
#include <Servo.h>

const int LED_ROJO = 4;
const int LED_VERDE = 5;
const int LED_AMARILLO = 6;
const int SERVO_PIN = 9;

Servo miServo;

void setup() {
  Serial.begin(9600);
  miServo.attach(SERVO_PIN);
  pinMode(LED_ROJO, OUTPUT);
  pinMode(LED_VERDE, OUTPUT);
  pinMode(LED_AMARILLO, OUTPUT);
  // estado inicial: puerta cerrada (servo 90), rojo encendido
  digitalWrite(LED_ROJO, HIGH);
  digitalWrite(LED_VERDE, LOW);
  miServo.write(90);
  Serial.println("SYSTEM:READY");
}

String readLine() {
  static String line = "";
  while (Serial.available()) {
    char c = Serial.read();
    if (c == '\n') {
      String out = line;
      line = "";
      return out;
    } else if (c != '\r') {
      line += c;
    }
  }
  return "";
}

void handleCommand(String cmd) {
  cmd.trim();
```

```
  if (cmd.length() == 0) return;

  Serial.print("RECV:"); Serial.println(cmd); // para debug

  if (cmd == "LIGHT:ON") {
    digitalWrite(LED_AMARILLO, HIGH);
    Serial.println("LIGHT:ON:OK");
  } else if (cmd == "LIGHT:OFF") {
    digitalWrite(LED_AMARILLO, LOW);
    Serial.println("LIGHT:OFF:OK");
  } else if (cmd == "DOOR:OPEN") {
  miServo.write(90); // abre (antes estaba 0)
  digitalWrite(LED_VERDE, HIGH);
  digitalWrite(LED_ROJO, LOW);
  Serial.println("DOOR:OPEN:OK");
  } else if (cmd == "DOOR:CLOSE") {
    miServo.write(0); // cierra (antes estaba 90)
    digitalWrite(LED_ROJO, HIGH);
    digitalWrite(LED_VERDE, LOW);
    Serial.println("DOOR:CLOSE:OK");
  } else if (cmd == "STATUS?") {
    String s = "STATUS:LIGHT=";
    s += (digitalRead(LED_VERDE) == HIGH ? "ON" : "OFF");
    s += ";DOOR=";
    s += (digitalRead(LED_ROJO) == HIGH ? "CLOSED" : "OPEN");
    Serial.println(s);
  } else {
    Serial.println("ERR:UNKNOWN_CMD");
  }
}

void loop() {
  String ln = readLine();
  if (ln.length() > 0) handleCommand(ln);
}
```

Commands

```
/*Command.java*/
package com.estef.smarthome.commands;

public interface Command {
```

```
    void execute();

    String serialize();
}
```

```
/*DoorCloseCommand.java*/

package com.estef.smarthome.commands;

import com.estef.smarthome.SerialManager;

public class DoorCloseCommand implements Command {
    private final SerialManager serial;

    public DoorCloseCommand(SerialManager s) {
        this.serial = s;
    }

    @Override
    public void execute() {
        serial.send("DOOR:CLOSE");
    }

    @Override
    public String serialize() {
        return "DOOR:CLOSE";
    }
}
```

```
/*DoorOpenCommand.java*/
package com.estef.smarthome.commands;

import com.estef.smarthome.SerialManager;

public class DoorOpenCommand implements Command {
    private final SerialManager serial;

    public DoorOpenCommand(SerialManager s) {
        this.serial = s;
    }

    @Override
```

```
    public void execute() {
        serial.send("DOOR:OPEN");
    }

    @Override
    public String serialize() {
        return "DOOR:OPEN";
    }
}
```

```
/*LightOffCommand.java*/
package com.estef.smarthome.commands;

import com.estef.smarthome.SerialManager;

public class LightOffCommand implements Command {
    private final SerialManager serial;

    public LightOffCommand(SerialManager s) {
        this.serial = s;
    }

    @Override
    public void execute() {
        serial.send("LIGHT:OFF");
    }

    @Override
    public String serialize() {
        return "LIGHT:OFF";
    }
}
```

```
/*LightOnCommand.java*/
package com.estef.smarthome.commands;

import com.estef.smarthome.SerialManager;

public class LightOnCommand implements Command {
    private final SerialManager serial;

    public LightOnCommand(SerialManager s) {
```

```
        this.serial = s;
    }

    @Override
    public void execute() {
        serial.send("LIGHT:ON");
    }

    @Override
    public String serialize() {
        return "LIGHT:ON";
    }
}
```

Factory

```
/*CommandFactory.java*/
package com.estef.smarthome.factory;

import com.estef.smarthome.SerialManager;
import com.estef.smarthome.commands.*;

public class CommandFactory {
    public static Command create(String type, SerialManager s) {
        switch (type) {
            case "LIGHT_ON":
                return new LightOnCommand(s);
            case "LIGHT_OFF":
                return new LightOffCommand(s);
            case "DOOR_OPEN":
                return new DoorOpenCommand(s);
            case "DOOR_CLOSE":
                return new DoorCloseCommand(s);
            default:
                return null;
        }
    }
}
```

State

```java
/*DoorClosed.java*/
package com.estef.smarthome.state;

public class DoorClosed implements DoorState {
    @Override
    public DoorState toggle() {
        return new DoorOpen();
    }

    @Override
    public String name() {
        return "CERRADA";
    }
}
```

```java
/*DoorOpen.java*/
package com.estef.smarthome.state;
public class DoorOpen implements DoorState {
    @Override
    public DoorState toggle() {
        return new DoorClosed();
    }

    @Override
    public String name() {
        return "ABIERTA";
    }
}
```

```java
/*DoorState.java*/
package com.estef.smarthome.state;

public interface DoorState {
    DoorState toggle(); // devuelve nuevo estado

    String name();
}
```

Strategy

```java
/*FixedMinutesStrategy.java*/
package com.estef.smarthome.strategy;

public class FixedMinutesStrategy implements ThresholdStrategy {
    private final long limit;

    public FixedMinutesStrategy(int minutes) {
        this.limit = minutes * 60L * 1000L;
    }

    @Override
    public boolean shouldWarn(long millisSinceOn) {
        return millisSinceOn >= limit;
    }
}
```

```java
/*ThresholdStrategy.java*/
package com.estef.smarthome.strategy;

public interface ThresholdStrategy {
    boolean shouldWarn(long millisSinceOn);
}
```

TextToSpeech

```java
package com.estef.smarthome.voice;

import java.io.IOException;

public class TextToSpeech {
    private boolean available = false;
    private String voiceCommand;

    public TextToSpeech() {
        checkAvailability();
    }

    private void checkAvailability() {
        // Verificar diferentes comandos de voz según el SO
```

```
        String os = System.getProperty("os.name").toLowerCase();

        if (os.contains("win")) {
            // Windows
            voiceCommand = "espeak";
            available = checkCommand(voiceCommand);
            if (!available) {
                // Intentar con PowerShell
                voiceCommand = "powershell -Command \"Add-Type -AssemblyName
System.Speech; (New-Object
System.Speech.Synthesis.SpeechSynthesizer).Speak('test')\"";
                available = checkCommand("powershell");
            }
        } else if (os.contains("nix") || os.contains("nux") ||
os.contains("mac")) {
            // Linux/Unix/macOS
            voiceCommand = "espeak";
            available = checkCommand(voiceCommand);
            if (!available) {
                voiceCommand = "say";
                available = checkCommand(voiceCommand);
            }
        }

        System.out.println("TTS disponible: " + available + ", comando: " +
voiceCommand);
    }

    private boolean checkCommand(String cmd) {
        try {
            Process process;
            if (cmd.equals("espeak")) {
                process = Runtime.getRuntime().exec(new String[] { "espeak", "--
version" });
            } else if (cmd.equals("say")) {
                process = Runtime.getRuntime().exec(new String[] { "say", "--
version" });
            } else if (cmd.equals("powershell")) {
                process = Runtime.getRuntime().exec(new String[] { "powershell",
"-Command", "exit" });
            } else {
                return false;
            }
            int exitCode = process.waitFor();
```

```
            return (exitCode == 0 || exitCode == 1); // Algunos comandos retornan
1 pero funcionan
        } catch (Exception e) {
            return false;
        }
    }

    public void speak(String text) {
        if (!available || text == null || text.trim().isEmpty()) {
            System.out.println("TTS (fallback): " + text);
            return;
        }

        try {
            String[] cmd;
            if (voiceCommand.equals("espeak")) {
                cmd = new String[] { "espeak", "-v", "es", "-s", "150", text };
            } else if (voiceCommand.equals("say")) {
                cmd = new String[] { "say", text };
            } else if (voiceCommand.startsWith("powershell")) {
                // Usar PowerShell en Windows
                String psCommand = String.format(
                        "Add-Type -AssemblyName System.Speech; $speak = New-
Object System.Speech.Synthesis.SpeechSynthesizer; $speak.Speak(\\\"%s\\\")",
                        text.replace("\"", "\\\""));
                cmd = new String[] { "powershell", "-Command", psCommand };
            } else {
                System.out.println("TTS (fallback): " + text);
                return;
            }

            ProcessBuilder pb = new ProcessBuilder(cmd);
            pb.start();
        } catch (IOException e) {
            System.err.println("Error en TTS: " + e.getMessage());
            System.out.println("TTS (fallback): " + text);
        }
    }

    public boolean isAvailable() {
        return available;
    }
}
```

Main

```
package com.estef.smarthome;

import javax.swing.*;
import java.awt.*;
import com.estef.smarthome.factory.CommandFactory;
import com.estef.smarthome.commands.Command;
import com.estef.smarthome.state.*;
import com.estef.smarthome.strategy.*;
import com.estef.smarthome.voice.TextToSpeech;
import com.estef.smarthome.voice.VoiceRecognizer;
import com.estef.smarthome.voice.VoiceListenerCallback;
import com.estef.smarthome.voice.VoiceCommandMapper;

public class MainGUI {
    private JFrame frame;
    private JTextArea log;
    private SerialManager serial;
    private DoorState doorState = new DoorClosed();
    private long lightOnTimestamp = -1;
    private ThresholdStrategy threshold = new FixedMinutesStrategy(1); //
1 min
    private VoiceRecognizer voiceRecognizer;


    private TextToSpeech tts = new TextToSpeech();

    public MainGUI() {
        serial = new SerialManager();
        voiceRecognizer = new VoiceRecognizer("Patrones-
SmartHome\\smarthome\\models\\vosk-model-small-es-0.42");
        initUI();
        initSerialListener();
        startLightChecker();

        // Verificar estado del TTS
        if (tts.isAvailable()) {
            append("Sistema de voz inicializado correctamente");
        } else {
            append("Sistema de voz no disponible - usando modo consola");
        }
    }
```

```
        private void initUI() {
            frame = new JFrame("SmartHome - Panel de Control");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(780, 500);
            frame.setLocationRelativeTo(null);
            frame.setLayout(new BorderLayout(10, 10));

            // ---- ENCABEZADO ----
            JLabel title = new JLabel("Smart Home - Sistema de
Automatización", SwingConstants.CENTER);
            title.setFont(new Font("Segoe UI", Font.BOLD, 22));
            title.setForeground(new Color(0, 90, 150));
            frame.add(title, BorderLayout.NORTH);

            // ---- PANEL DE CONEXIÓN ----
            JPanel top = new JPanel(new FlowLayout(FlowLayout.CENTER, 10,
10));
            top.setBackground(new Color(240, 248, 255));
            top.setBorder(BorderFactory.createTitledBorder("Conexión"));

            String[] ports = SerialManager.listPorts();
            JComboBox<String> cbPorts = new JComboBox<>(ports);
            JButton btnConnect = createButton("Conectar",
"icons/connect.png");

            top.add(new JLabel("Puerto disponible:"));
            top.add(cbPorts);
            top.add(btnConnect);

            // ---- PANEL DE CONTROLES ----
            JPanel center = new JPanel(new GridLayout(2, 3, 15, 15));
            center.setBackground(Color.WHITE);
            center.setBorder(BorderFactory.createTitledBorder("Controles"));

            JButton bLightOn = createButton("Encender Luz",
"icons/light_on.png");
            JButton bLightOff = createButton("Apagar Luz",
"icons/light_off.png");
            JButton bDoorOpen = createButton("Abrir Puerta",
"icons/door_open.png");
            JButton bDoorClose = createButton("Cerrar Puerta",
"icons/door_close.png");
```

```
            JButton bVoice = createButton("Comando por voz", "icons/mic.png");


            center.add(bLightOn);
            center.add(bLightOff);
            center.add(bDoorOpen);
            center.add(bDoorClose);
            center.add(bVoice);

            // ---- PANEL DE REGISTRO (CONSOLA) ----
            log = new JTextArea();
            log.setFont(new Font("Consolas", Font.PLAIN, 13));
            log.setEditable(false);
            log.setBackground(new Color(245, 245, 245));
            log.setBorder(BorderFactory.createLineBorder(Color.LIGHT_GRAY));

            JScrollPane scrollPane = new JScrollPane(log);
            scrollPane.setPreferredSize(new Dimension(740, 140));
            scrollPane.setBorder(BorderFactory.createTitledBorder("Registro de
Actividades"));

            // ---- PANEL DE PIE DE PÁGINA ----
            JPanel footerPanel = new JPanel(new BorderLayout());
            footerPanel.setBackground(Color.WHITE);
            footerPanel.setBorder(BorderFactory.createEmptyBorder(5, 10, 5,
10));

            JLabel footer = new JLabel(
                    "<html><center>" +
                            "Proyecto A198 - Patrones de Software<br>" +
                            "Integrantes: Iván Darío Hernández, Estefanía
Moreno Reyes, Edson Jaime Pinto Abreo, " +
                            "Daniel José Suárez Gonzales, Jesús David Arias
Estupiñán<br>" +
                            "Empresa: Soluciones Digitales Daniel Tomas |
Sector: Tecnología - Domótica" +
                            "</center></html>",
                    SwingConstants.CENTER);
            footer.setFont(new Font("Segoe UI", Font.PLAIN, 12));
            footer.setForeground(Color.DARK_GRAY);
            footerPanel.add(footer, BorderLayout.CENTER);
```

```
        // ---- ENSAMBLE ----
        JPanel mainPanel = new JPanel(new BorderLayout(10, 10));
        mainPanel.add(top, BorderLayout.NORTH);
        mainPanel.add(center, BorderLayout.CENTER);
        mainPanel.add(scrollPane, BorderLayout.SOUTH);

        frame.add(mainPanel, BorderLayout.CENTER);
        frame.add(footerPanel, BorderLayout.SOUTH);

        frame.setVisible(true);

        // ---- ACCIONES ----
        btnConnect.addActionListener(e -> {
            String p = (String) cbPorts.getSelectedItem();
            if (p == null) {
                append("No se detectaron puertos disponibles.");
                return;
            }
            if (serial.open(p, 9600)) {
                append("Conexión establecida correctamente con el puerto "
+ p);
                tts.speak("Sistema conectado");
            } else {
                append("Error: no fue posible conectar con el puerto " +
p);
                tts.speak("Error de conexión");
            }
        });

        bLightOn.addActionListener(e -> {
            Command c = CommandFactory.create("LIGHT_ON", serial);
            if (c != null) {
                c.execute();
                lightOnTimestamp = System.currentTimeMillis();
                append("Comando enviado: encender luz.");
                tts.speak("Encendiendo luz");
            }
        });

        bLightOff.addActionListener(e -> {
            Command c = CommandFactory.create("LIGHT_OFF", serial);
            if (c != null) {
                c.execute();
```

```
                    append("Comando enviado: apagar luz.");
                    lightOnTimestamp = -1;
                    tts.speak("Apagando luz");
                }
            });

            bDoorOpen.addActionListener(e -> {
                Command c = CommandFactory.create("DOOR_OPEN", serial);
                if (c != null) {
                    c.execute();
                    append("Comando enviado: abrir puerta.");
                    doorState = new DoorOpen();
                    append("Estado local puerta: " + doorState.name());
                    tts.speak("Abriendo puerta");
                }
            });

            bDoorClose.addActionListener(e -> {
                Command c = CommandFactory.create("DOOR_CLOSE", serial);
                if (c != null) {
                    c.execute();
                    append("Comando enviado: cerrar puerta.");
                    doorState = new DoorClosed();
                    append("Estado local puerta: " + doorState.name());
                    tts.speak("Cerrando puerta");
                }
            });
            bVoice.addActionListener(e -> {
        append("Iniciando reconocimiento por voz...");
        if (tts.isAvailable()) {
            tts.speak("Escuchando");
        }

        voiceRecognizer.startListening(new VoiceListenerCallback() {
            @Override
            public void onTextRecognized(String text) {
                SwingUtilities.invokeLater(() -> {
                    append("Reconocido: \"" + text + "\"");

                    String cmdKey = VoiceCommandMapper.map(text);
                    if (cmdKey == null) {
                        append("No se reconoció un comando válido.");
                        if (tts.isAvailable()) tts.speak("No entendí el
comando.");
```

```
                        return;
                    }

                    Command c = CommandFactory.create(cmdKey, serial);
                    if (c == null) {
                        append("No se pudo crear el comando para: " + cmdKey);
                        if (tts.isAvailable()) tts.speak("Error al ejecutar el
comando.");
                        return;
                    }

                    c.execute();
                    append("Comando ejecutado por voz: " + cmdKey);

                    switch (cmdKey) {
                        case "LIGHT_ON":
                            lightOnTimestamp = System.currentTimeMillis();
                            tts.speak("Encendiendo luz");
                            break;
                        case "LIGHT_OFF":
                            lightOnTimestamp = -1;
                            tts.speak("Apagando luz");
                            break;
                        case "DOOR_OPEN":
                            doorState = new DoorOpen();
                            append("Estado local puerta: " +
doorState.name());
                            tts.speak("Abriendo puerta");
                            break;
                        case "DOOR_CLOSE":
                            doorState = new DoorClosed();
                            append("Estado local puerta: " +
doorState.name());
                            tts.speak("Cerrando puerta");
                            break;
                    }
                });
            }

            @Override
            public void onError(String message) {
                SwingUtilities.invokeLater(() -> {
                    append("Voz: " + message);
                });
```

```
            }
        });
    });


        }

        private JButton createButton(String text, String iconPath) {
            JButton button = new JButton(text);
            button.setFont(new Font("Segoe UI", Font.BOLD, 14));
            button.setBackground(new Color(225, 235, 250));
            button.setFocusPainted(false);
            button.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
            button.setBorder(BorderFactory.createLineBorder(new Color(190,
200, 230)));
            button.setIcon(loadIcon(iconPath));
            button.setHorizontalAlignment(SwingConstants.LEFT);
            return button;
        }

        private ImageIcon loadIcon(String path) {
            try {
                ImageIcon icon = new ImageIcon(getClass().getResource(path));
                Image img = icon.getImage().getScaledInstance(20, 20,
Image.SCALE_SMOOTH);
                return new ImageIcon(img);
            } catch (Exception e) {
                return null;
            }
        }

        private void startLightChecker() {
            Timer t = new Timer(30_000, e -> {
                if (lightOnTimestamp > 0) {
                    long elapsed = System.currentTimeMillis() -
lightOnTimestamp;
                    if (threshold.shouldWarn(elapsed)) {
                        String mensaje = "Aviso: la luz ha permanecido
encendida por un tiempo prolongado.";
                        append(mensaje);

                        // AVISO POR VOZ
```

```
                        tts.speak("Atención. Luz encendida por mucho
tiempo.Por favor, apague la luz . Gracias");

                        JOptionPane.showMessageDialog(
                                frame,
                                mensaje,
                                "Advertencia de Luz Encendida",
                                JOptionPane.WARNING_MESSAGE);
                        lightOnTimestamp = -1;
                    }
                }
            });
            t.start();
        }

        private void initSerialListener() {
            serial.addListener(msg -> SwingUtilities.invokeLater(() -> {
                append("Arduino -> " + msg);
                if (msg.startsWith("LIGHT:ON:OK")) {
                    append("Confirmado: luz encendida");
                    tts.speak("Luz encendida confirmada");
                } else if (msg.startsWith("LIGHT:OFF:OK")) {
                    append("Confirmado: luz apagada");
                    tts.speak("Luz apagada confirmada");
                } else if (msg.startsWith("DOOR:OPEN:OK")) {
                    append("Confirmado: puerta abierta");
                    tts.speak("Puerta abierta confirmada");
                } else if (msg.startsWith("DOOR:CLOSE:OK")) {
                    append("Confirmado: puerta cerrada");
                    tts.speak("Puerta cerrada confirmada");
                } else if (msg.startsWith("ERR:")) {
                    append("Error del sistema: " + msg);
                    tts.speak("Error en el sistema");
                }
            }));
        }

        private void append(String s) {
            log.append(s + "\n");
            log.setCaretPosition(log.getDocument().getLength());
        }

        public static void main(String[] args) {
            SwingUtilities.invokeLater(MainGUI::new);
```

```
        }
    }
```

Serial Manager

```
package com.estef.smarthome;

import com.fazecast.jSerialComm.SerialPort;
import java.io.*;
import java.util.*;
import java.util.concurrent.CopyOnWriteArrayList;

public class SerialManager {
    private SerialPort port;
    private BufferedReader in;
    private PrintWriter out;
    private Thread readerThread;

    public interface Listener {
        void onMessage(String msg);
    }

    private List<Listener> listeners = new CopyOnWriteArrayList<>();

    public void addListener(Listener l) {
        listeners.add(l);
    }

    public void removeListener(Listener l) {
        listeners.remove(l);
    }

    public boolean open(String portName, int baud) {
        port = SerialPort.getCommPort(portName);
        port.setBaudRate(baud);
        if (!port.openPort())
            return false;
        try {
            in = new BufferedReader(new
InputStreamReader(port.getInputStream()));
            out = new PrintWriter(port.getOutputStream(), true);
        } catch (Exception e) {
```

```
            e.printStackTrace();
            return false;
        }

        readerThread = new Thread(() -> {
            try {
                String line;
                while (!Thread.currentThread().isInterrupted() && (line =
in.readLine()) != null) {
                    for (Listener l : listeners)
                        l.onMessage(line);
                }
            } catch (IOException e) {
                /* cerrado */ }
        }, "SerialReader");
        readerThread.start();
        return true;
    }

    public void send(String s) {
        if (out != null) {
            out.print(s + "\n");
            out.flush();
            System.out.println("SerialManager -> Sent: " + s);
        }
    }

    public void close() {
        try {
            if (readerThread != null)
                readerThread.interrupt();
        } catch (Exception ignore) {
        }
        if (port != null && port.isOpen())
            port.closePort();
    }

    public static String[] listPorts() {
        SerialPort[] ps = SerialPort.getCommPorts();
        String[] arr = new String[ps.length];
        for (int i = 0; i < ps.length; i++)
            arr[i] = ps[i].getSystemPortName();
        return arr;
    }
}
```

Montaje Físico

El objetivo del montaje es implementar un sistema en Arduino UNO que nos permita:

- Encender y apagar luces (LEDs)
- Controlar el estado de una puerta (abrir/cerrar con el servomotor)
- Notificar al usuario los cambios de estado en el dispositivo con Aviso por voz

Materiales

- 1 Arduino UNO
- 1 Protoboard
- 3 Leds
- 3 Resistencias de 220 Ohmios (rojo, rojo, marrón, dorado)
- 1 Servomotor (SG90)
- Cables o puentes

Procedimiento

1. Conexión del Led Rojo

- Led Rojo: Ánodo (pata larga) → Resistencia 220Ω→Pin 4 Digital Arduino
- Cátodo (pata corta) → GND Arduino

2. Conexión del Led Verde

- Led Verde: Ánodo (pata larga) → Resistencia 220Ω → Pin 5 Digital Arduino
- Cátodo (pata corta) → GND Arduino

3. Conexión del Led Amarillo

- LED Amarillo: Ánodo (pata larga) → Resistencia 220Ω → Pin 6 Digital Arduino

- Cátodo (pata corta) → GND Arduino

4. Conexión del Servomotor (SG90)

- Cable Naranja/Marrón (Señal) → Pin 9 Digital Arduino
- Cable Rojo (VCC/+5V) → 5V Arduino
- Cable Marrón/Negro (GND) → GND Arduino

5. Conexiones de Tierra (GND) Comunes

Crear un bus de GND en el protoboard:

- Conectar todos los cátodos de LEDs al mismo bus GND
- Conectar el GND del servomotor al mismo bus o en el GND del Arduino
- Del bus GND conectar un cable al GND de Arduino

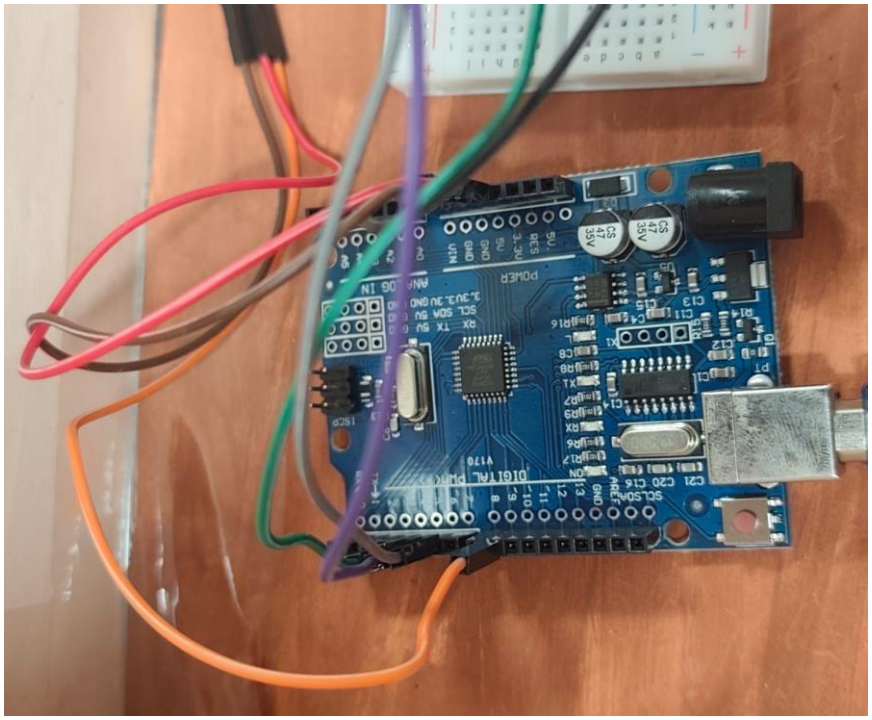


Figura 1: Conexión en el Arduino

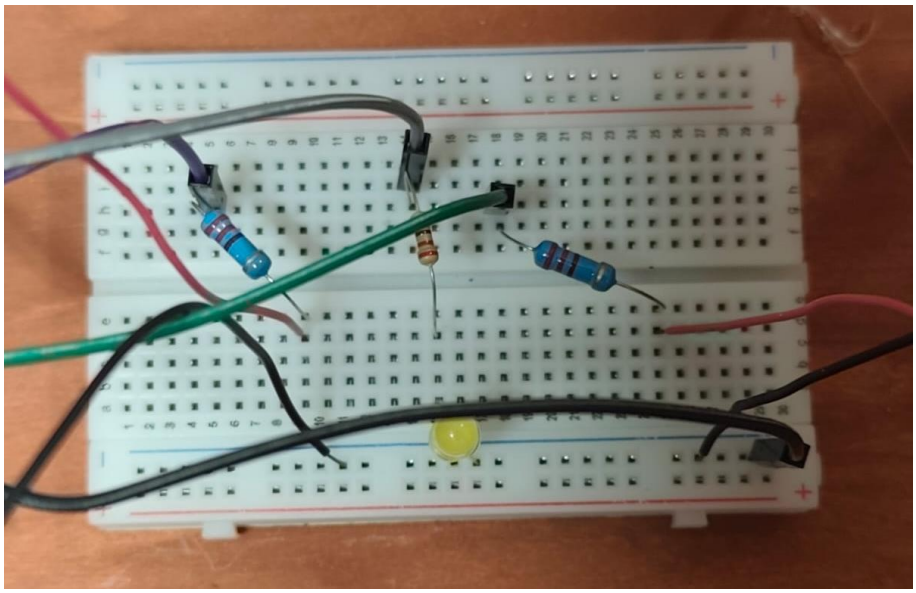


Figura 2: Conexión en el Protoboard

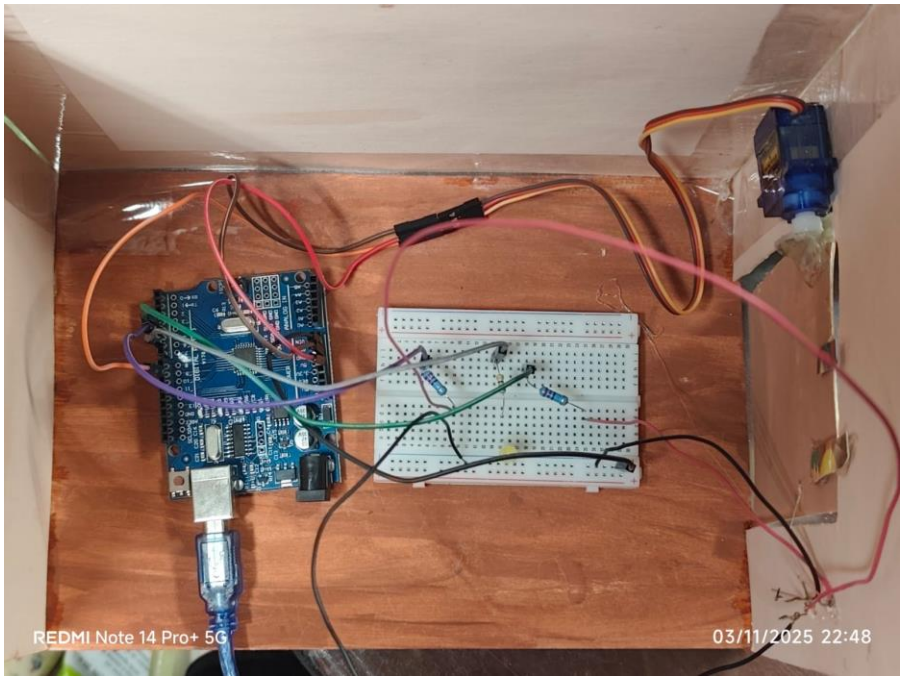


Figura 3: Conexión General

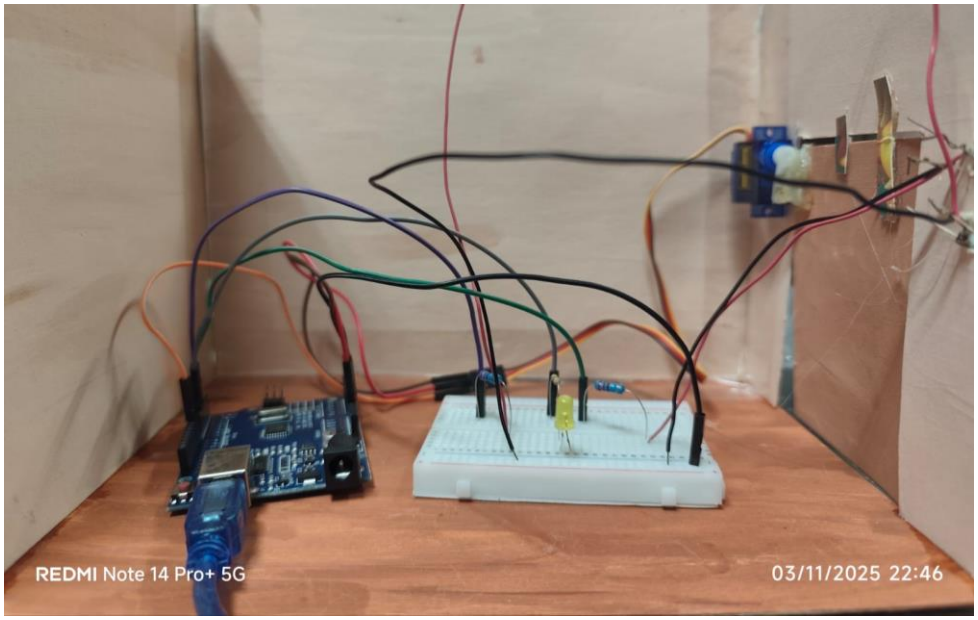


Figura 4: Conexión Interna General

Figura 5: Conexión Interna General

Figura 6: Casa Inteligente

SmartHome - Panel de Control
Smart Home - Sistema de Automatización
Conexión
Puerto disponible:
COM3
Conectar
Controles
Encender Luz
Apagar Luz
Abrir Puerta
Cerrar Puerta
Comando por voz
Registro de Actividades
Sistema de voz inicializado correctamente
Proyecto A198 - Patrones de Software
Integrantes: Iván Darío Hernández, Estefanía Moreno Reyes, Edson Jaime Pinto Abreo, Daniel José Suárez Gonzales, Jesús David Arias Estupiñán
Empresa: Soluciones Digitales Daniel Tomas | Sector: Tecnología - Domótica

Figura 7: Interfaz

Prueba de Funcionamiento

Link:

https://www.canva.com/design/DAG34ktygCA/k-MbN2ONmdZ6Wurg79oSJw/watch?utm_content=DAG34ktygCA&utm_campaign=designshare&utm_medium=link2&utm_source=uniquelinks&utlId=h28cff0ab27

Conclusiones

1. La aplicación estratégica de los patrones de diseño (Command, Factory, Observer, State) fue fundamental para **estructurar el software de manera organizada y modular**. Cada patrón resolvió un desafío específico del diseño (por ejemplo, el patrón **Command** para encapsular acciones y **Observer** para las notificaciones al usuario), resultando en un código con **responsabilidades claramente separadas**, lo que maximiza la reusabilidad, la flexibilidad y facilita la integración de nuevos dispositivos en el futuro
2. El sistema opera bajo un modelo de control **híbrido**, donde la lógica de negocio, la interfaz gráfica (GUI en Java), el reconocimiento de voz (`VoiceRecognizer`) y el Text-to-Speech (TTS) residen en el host , mientras que el control físico de los actuadores (LEDs y servomotor) reside en el microcontrolador (Arduino). La **comunicación serial** actúa como el protocolo de bajo nivel para el intercambio de comandos y confirmaciones de estado entre ambos entornos
3. La integración de componentes físicos (Arduino, LEDs y servomotor) con una arquitectura de software basada en patrones de diseño permitió evidenciar cómo la combinación de hardware accesible y buenas prácticas de programación puede dar lugar a sistemas confiables y fáciles de escalar. Esta experiencia demuestra que un prototipo de Smart Home puede desarrollarse de manera económica sin comprometer la calidad del diseño ni la claridad en la organización del código.
4. El uso del reconocimiento de voz y la retroalimentación auditiva mediante Text-to-Speech amplió la interacción del usuario con el sistema, logrando una experiencia más intuitiva y moderna. Esta funcionalidad, además de mejorar la accesibilidad del prototipo, evidencia el potencial de integrar interfaces naturales en futuros proyectos de domótica, abriendo la puerta a sistemas más inclusivos, autónomos y eficientes.

Referencias

Flores, L. E. (2014). *Analisis y Diseno de un Sistema Domotico.* Obtenido de https://repositorio.puce.edu.ec/server/api/core/bitstreams/26469a9d-86d1-4e4e-bd2e-2c8ec8fa6c49/content

Guailas, W. R. (2023). *Diseno y Simulacion de un Sistema Domotico para una Vivienda Inteligente con Protocolo Loxone.* Obtenido de https://dspace.ups.edu.ec/bitstream/123456789/24511/1/UPS-CT010389.pdf